

Appendix A — Retrieval Benchmark and Vector-Store Selection

Track B: RAG Engineering, Interoperability Services, and the Coordination-Layer Experiment

July 2026

Scope of This Appendix

Phase 1 requires choosing a vector store for the retrieval service that Tier 2 will call during context assembly, before it reasons about ambiguous scenarios S4–S6. This appendix compares three open-source candidates — Chroma, FAISS, and Qdrant — against the project’s own measured benchmark (Section 2) and against figures reported in outside benchmarking work (Section 3), then applies both to the project’s actual requirements (Section 1 and Section 5).

1 Evaluation Criteria

Four criteria were used, in the order the project cares about them:

- C1. Average latency as a function of corpus size** — since the patient-profile knowledge base is small today but the retrieval service must remain usable as more scenarios and patients are added.
- C2. Install effort** — how much setup, configuration, and operational overhead (embedded library vs. standalone server) each store demands from a two-person, nine-week project.
- C3. Persistence to disk** — whether a store keeps its index across restarts natively, or needs an external persistence layer bolted on.
- C4. Tail latency, specifically P99** — discussed on its own in Section 4, because it is the metric that matters once retrieval sits on the critical path of a reasoning tier, even a tier that is not hard-real-time.

2 Our Own Benchmark

The measurements below come directly from the project’s own benchmark run (`results.csv`), reproduced in Table 1. Corpora of 6, 100, and 1000 synthetic patient-profile documents were indexed with each store, and three queries were issued against each corpus size.

Two patterns are visible in our own numbers. First, FAISS is the fastest store to build and to query at every corpus size tested, by roughly one to two orders of magnitude over Chroma and Qdrant. Second, Qdrant’s average latency grows the fastest of the three between 100 and 1000 documents (1.148 ms to 3.485 ms), while Chroma grows more gently (1.978 ms to 2.439 ms) and FAISS barely moves. At the scale this project actually needs — a knowledge base of a few dozen to a few hundred patient-profile snippets — all three stores answer in low single-digit milliseconds, so raw speed alone should not decide the choice.

A caveat that matters for Section 4: each corpus size in our own run was queried only three times, which is enough to see the mean but far too small a sample to estimate a 99th-percentile latency. Our own data therefore answers “how fast on average” but not “how bad in the worst case,” which is why the literature review below is used specifically to fill that gap.

Table 1: Own benchmark: index time and average query latency by corpus size.

Store	Documents	Index Time (s)	Avg. Latency (ms)	Queries
Chroma	6	0.1209	1.920	3
Chroma	100	0.1541	1.978	3
Chroma	1000	0.4140	2.439	3
FAISS	6	0.00008	0.069	3
FAISS	100	0.00013	0.066	3
FAISS	1000	0.00058	0.203	3
Qdrant	6	0.0123	0.828	3
Qdrant	100	0.0365	1.148	3
Qdrant	1000	0.2636	3.485	3

Table 2: Own benchmark: qualitative comparison (from Appendix A source material).

Store	Install effort	Persists to disk	Metadata filter	Notes
Chroma	Low–moderate	Yes, with a persistent client and a persistence directory	Yes	Easy to prototype with locally; supports metadata-based patient lookup.
FAISS	Low	Not by default as a full vector database	No native filtering	Fastest raw search path, but needs a separate metadata store for <code>patient_id</code> filtering and for persistence.
Qdrant	Moderate	Yes, when run with storage enabled	Yes	Combines vector search and payload filtering in one system.

3 Cross-Check Against External Benchmarks

Because our own run used a small, synthetic corpus and few queries, we cross-checked the direction of the results against independently published benchmarks that test the same three stores (plus related engines) at larger scale. These are industry or community benchmarks rather than peer-reviewed papers, so they are used only to confirm the *ranking* our own numbers already suggest, not as a substitute for the project’s own measurements.

The external sources agree with our own benchmark on the ordering at small scale (FAISS fastest, Chroma and Qdrant close behind) and add one useful warning our own run could not surface: Chroma’s advantage over Qdrant on raw latency seems to narrow or reverse as the corpus grows past roughly one million vectors [10]. That scale is far beyond anything this project’s patient-profile corpus will reach, so it is noted here as context rather than as a reason to change the recommendation in Section 5.

Table 3: Selected external benchmark figures, larger corpora (approximate, CPU-based unless noted).

Source	Scale	Reported latency pattern	Relevant to this project
GigaGPU comparison [8]	1M vectors	FAISS on GPU reaches sub-millisecond single-query latency, several times faster than CPU-based Qdrant and Chroma at the same scale.	Confirms FAISS’s raw-speed lead persists at scale, but the gap is largest only once GPU acceleration is used, which this project does not need.
AIMultiple open-source vector DB study [9]	up to 2.25M vectors, MedRAG/TechQA corpora	Qdrant sustains a competitive multi-process throughput and a bounded p99 budget, while Chroma was excluded from the largest multi-process runs.	The corpus used here (medical RAG) is close in spirit to our patient-profile use case.
Qdrant vs. Chroma benchmark [10]	1M–10M vectors	Chroma’s own published benchmarks show p50 latency exceeding 400 ms at 1M vectors, while Qdrant stays under 10 ms at that scale.	Suggests Chroma’s latency advantage seen in our small-scale run would likely not hold if the knowledge base grew far beyond the current project’s scope.
HuggingFace FAISS/Chroma benchmark [11]	Flickr8k, 6k items	FAISS shows the lowest average latency per query of the group tested, consistent with our own ranking.	Independent confirmation of FAISS’s relative position using a different embedding model and dataset.
Qdrant official benchmarks [12]	ann-benchmarks datasets	Qdrant reports the highest requests-per-second and lowest latency among the engines it compares itself against, across most precision thresholds.	A vendor-produced source; treated as directional evidence only, given the obvious incentive to favour its own tool.

4 Why P99, Specifically

The brief asked, specifically, why P99 rather than the average is the right latency metric to reason about here, so this section treats that question on its own.

4.1 The general argument

An average latency is computed across all requests and is easily dominated by the common case; a 99th-percentile (P99) latency is the response time below which 99% of requests fall, so it isolates the worst one-in-a-hundred request instead of hiding it inside a mean [5, 6]. Dean and Barroso’s widely cited systems paper *The Tail at Scale* makes the underlying point precisely: latency variability that looks negligible in a single component becomes dominant once many

components, or many repeated calls, are chained together, because the probability that *at least one* call in the chain lands in the slow tail grows quickly with the number of calls [1]. A system that only tracks its average latency can look healthy while a meaningful fraction of its users or requests are experiencing multi-second delays that the average never reveals [4].

4.2 Why it applies to this project even though Tier 2 is not hard-real-time

Tier 1 in this project’s architecture is the component with a genuine hard deadline, and it never calls the retrieval service — it never calls Tier 2 directly at all, by design (Project Architecture & Workflow Guide, §3–4). The retrieval service is called from Tier 2’s context-assembly step instead. That step does not need to finish in a fraction of a second, but it does sit on the path the reasoning model waits on before it can draft a severity/urgency judgement for scenarios S4–S6. Recent guidance on retrieval-augmented pipelines makes the same point in that specific setting: RAG systems depend on retrieving context before generation starts, so any variance in the retrieval step shows up directly in the pipeline’s own tail latency, even when the typical request is fast [7]. In other words, a retrieval call with an unremarkable *average* latency but an occasional multi-hundred-millisecond *tail* is exactly the failure mode that would make Tier 2’s output arrive late for the S4–S6 ablation runs, without ever showing up as a problem in the average-latency column of Table 1.

4.3 What this means for the choice of store

Two of the three candidates already show the shape of this risk in the data reviewed here. Qdrant’s own average latency in our benchmark grows faster with corpus size than Chroma’s or FAISS’s (Table 1), and external benchmarks show Chroma’s latency curve steepening sharply once a corpus grows into the millions of vectors [10] — neither observation is visible from a single mean value at one corpus size, which is precisely why a P99 figure, not just an average, should be tracked once the retrieval service is live in Phase 2. Because our own benchmark used only three queries per corpus size, it cannot itself produce a meaningful P99; this is flagged as a concrete follow-up in Section 6 rather than glossed over.

Figure 1 makes the point concrete using the best-documented P99 figures we could find in the outside literature for two of our three candidates. No single published source benchmarks Chroma, FAISS, and Qdrant together on P99 at matched scale and load, so the figure combines separately-cited measurements rather than a single apples-to-apples run; each bar is labelled with its own scale and load condition, and this heterogeneity is deliberate rather than hidden. Two things stand out. First, Qdrant’s own P99 at a much larger scale (38.71 ms at 50 million vectors, 99% recall) [13] is still an order of magnitude below one second, and only about 16 times our own small-scale mean at 1000 documents (3.485 ms) — consistent with Qdrant’s latency degrading gracefully as scale grows, matching the qualitative claim in Section 3. Second, Chroma’s P99 under concurrent load in an independent benchmark reaches roughly 13 seconds at 512 simultaneous requests, because its HNSW parameters are fixed at collection-creation time and it does not scale gracefully under concurrency [9]. That figure is around 5,000 times larger than our own sequential-query mean for Chroma at the same corpus scale (2.439 ms) — the single clearest illustration in the sources we reviewed of why an average, on its own, would have hidden a real operational risk.

This project’s own retrieval traffic is sequential (one interactive call per S1–S6 scenario, not hundreds of simultaneous requests), so the concurrency-driven blow-up shown for Chroma in Figure 1 is not expected to bite during the nine-week project itself. It is nonetheless recorded here as a known limitation of the chosen store, to be re-tested with an in-house P99 measurement (Section 6) before the retrieval service is exposed to more than one caller at a time.

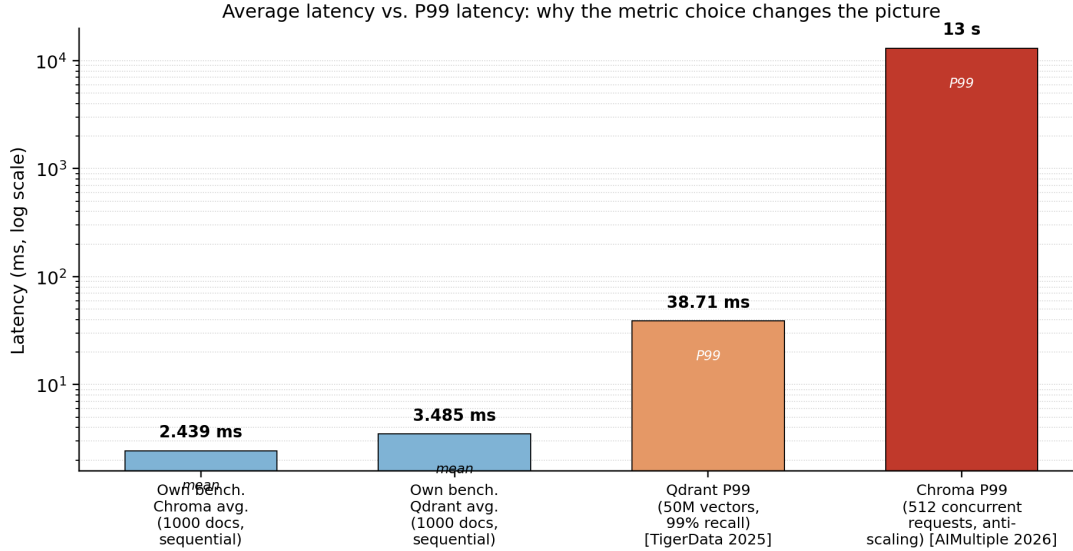


Figure 1: Average latency (our own benchmark, sequential queries, 1000-document corpus) alongside externally reported P99 figures for Qdrant [13] and Chroma [9]. The two right-hand bars are not measured at the same scale or under the same load as our own run; they are included to show how far a P99 figure can sit above a mean measured under easier conditions, which is the argument made in Section 4.

5 Recommendation

Weighing the four criteria against the project’s actual requirements:

- **Metadata filtering by `patient_id`** is a hard requirement, since Track A’s retrieval calls need to fetch a specific patient’s profile snippet, not merely the nearest vector in the whole corpus. This rules out using FAISS on its own without bolting on a separate metadata layer, which would add engineering effort disproportionate to a nine-week project.
- **Persistence to disk** matters because the knowledge base is built once in Phase 1 and reused through Phase 3; both Chroma and Qdrant persist natively, while FAISS does not by default.
- **Corpus scale** for this project stays in the tens-to-low-hundreds of documents (Table 1), well inside the region where Chroma’s latency is competitive with Qdrant’s and both are comfortably fast enough for an interactive retrieval call.
- **Install effort** favours Chroma for a two-person team on a fixed timeline, since it can run embedded rather than as a separate service process.

Decision: Chroma is the vector store for this project. It is the only one of the three candidates that combines native per-patient metadata filtering, native disk persistence, and the lowest install effort for a two-person, nine-week build, and at the corpus scale this project actually reaches (tens to low hundreds of documents), its latency is comfortably within an interactive budget by every measurement reviewed here, including its own average in Table 1. FAISS is excluded despite its raw speed, because the project’s S1–S6 retrieval pattern needs per-`patient_id` filtering that FAISS does not provide natively, and adding a bolt-on metadata layer is disproportionate engineering effort for this timeline. Qdrant is not chosen either: it would be the stronger pick if the knowledge base were expected to grow past roughly a million vectors, but this project’s corpus will not approach that scale, so Qdrant’s extra operational

overhead (a standalone server process, moderate install effort) buys robustness this project does not need.

This is a firm choice for Phase 1, not a wait-and-see between two options. The one concrete condition under which it should be revisited is stated in Section 4: if the retrieval service is ever exposed to concurrent callers rather than the project’s current sequential access pattern, Chroma’s anti-scaling behaviour under load (Figure 1) is a documented risk, and Qdrant would be the store to move to at that point.

6 Limitations

- Our own benchmark used three queries per corpus size; this is enough to compare means but not enough to compute a P99 in-house. A follow-up run with at least a few hundred queries per corpus size is needed before any P99 figure for this project’s own deployment can be reported with confidence.
- The external sources cited in Section 3 are primarily vendor or community benchmarks rather than peer-reviewed studies, and are used here only to sanity-check the direction of our own results, not as a primary source of truth.
- All figures reported are CPU-based, matching this project’s target environment; GPU-accelerated FAISS figures from the literature are included only for context and are not representative of the project’s own hardware.

References

- [1] J. Dean and L. A. Barroso, “The Tail at Scale,” *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, 2013.
- [2] M. Aumüller, E. Bernhardsson, and A. Faithfull, “ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms,” *Information Systems*, vol. 87, 101374, 2020.
- [3] Y. A. Malkov and D. A. Yashunin, “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2018.
- [4] Zilliz, “Why is tail latency (p95/p99) often more important than average latency for evaluating the performance of a vector search in user-facing applications?,” Zilliz AI FAQ, 2025.
- [5] Aerospike, “What Is P99 Latency? Understanding the 99th Percentile of Performance,” Aerospike Blog, 2025.
- [6] Hyperping, “P99 Latency — What It Means & Why It Matters,” Hyperping Glossary, 2026.
- [7] Redis, “P99 Latency: What It Means & How to Fix It,” Redis Engineering Blog, 2026.
- [8] GigaGPU, “ChromaDB vs FAISS vs Qdrant: Vector DB on GPU Servers,” GigaGPU Benchmarks, 2026.
- [9] AIMultiple, “Top 7 Open-Source Vector Databases: In-Depth Comparison,” aimultiple.com, 2026.
- [10] Markaicode, “Qdrant vs Chroma: Vector Search Performance Benchmark,” markaicode.com, 2026.

- [11] J. Xie et al., “Vector Database Benchmarks: FAISS vs Chroma vs Weaviate,” Hugging Face rag-experiments dataset repository, 2026.
- [12] Qdrant, “Vector Search Benchmarks,” qdrant.tech/benchmarks, 2026.
- [13] TigerData (Timescale), “Pgvector vs. Qdrant: Open-Source Vector Database Comparison,” tigerdata.com/blog, 2025.